

Reference Notes

Week 2: Loop Control with Break and Continue

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

What strategies help us debug and optimize our programs? Sometimes you need to exit a loop early when you find what you're looking for. Other times, you want to skip certain iterations but keep the loop running. This week, we'll master the art of precise loop control with **break** and **continue** statements!

Prerequisites

Before starting this week, you should be comfortable with:

- Creating and using **while** loops (Week 1)
- Understanding loop conditions and when loops stop (Week 1)
- Using **if** statements inside loops (Module 1, Week 7)
- Using **for** loops with **range()** (Module 1, Week 9)

1.2 What You Already Know

You can create **while** loops that continue until a condition becomes false. You understand how to use special values and flag variables to control loops. You've practiced tracing through loops to predict their behavior. Now we'll add powerful new tools to control exactly when and how loops execute!

1.3 What You'll Be Able to Do

By the end of this week, you'll:

- Apply **break** statements to exit loops immediately when needed
- Use **continue** statements to skip iterations without stopping the loop
- Control program flow with precision using **break** and **continue** strategically
- Build robust menu systems and game loops
- Identify and fix infinite loops using systematic debugging techniques
- Understand when **break/continue** make code cleaner vs. more complex

2 VIDEO 1: The Break Statement

2.1 Understanding Break

The **break** statement is like an emergency exit for loops. When Python encounters **break**, it immediately jumps out of the current loop - no questions asked! This is perfect when you've found what you're looking for or when a special condition occurs.

2.2 Your First Break Statement

Note that the break statement works in both **for** and **while** loops. In the following example, we search for the first number divisible by 7 between 1 and 50. When we find it, we print it and use **break** to exit the loop immediately.

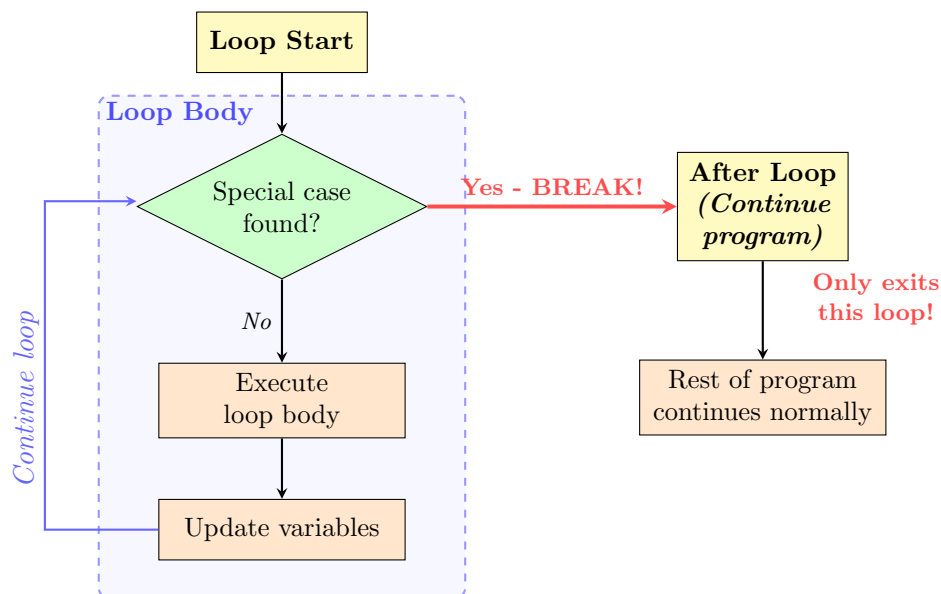
```

1 # Find the first number divisible by 7
2 for num in range(1, 50):
3     if num % 7 == 0:
4         print(f"Found it: {num}")
5         break #<----- Exit the loop immediately!
6     print(f"Checking {num}...")
7 print("Search complete")

```

2.3 How Break Works

Following is a flow diagram illustrating how the **break** statement affects loop execution.



2.4 Break in while Loops

You can also use **break** in **while** loops to exit based on a condition inside the loop body. This is especially useful for menu systems or game loops where you want to allow the user to quit at any time.

```

1 # Game loop with break for instant quit
2 score = 0
3 while True: # Infinite loop!
4     action = input("Enter command (play/quit): ")

```

```

5     if action == "quit":
6         print("Thanks for playing!")
7         break # Exit the while loop
8     elif action == "play":
9         score = score + 10
10        print(f"Score: {score}")
11    print("Game Over")

```

2.5 Break Only Exits Current Loop

When you have nested loops, `break` only exits the innermost loop:

```

1  # Break in nested loops
2  for i in range(3):
3      print(f"Outer loop: {i}")
4      for j in range(5):
5          if j == 2:
6              break # Only exits inner loop
7          print(f"    Inner: {j}")
8      print("    Inner loop done")
9  print("All done!")

```

When you run this, you'll see that when `j` reaches 2, it breaks out of the inner loop but continues with the next iteration of the outer loop. Following output illustrates this:

```

Outer loop: 0
  Inner: 0
  Inner: 1
  Inner loop done
Outer loop: 1
  Inner: 0
  Inner: 1
  Inner loop done
Outer loop: 2
  Inner: 0
  Inner: 1
  Inner loop done
All done!

```

Common Error

Common Mistake:

Break exits the entire program

Wrong thinking:

“When I use `break`, my whole program stops”

Reality:

Break only exits the current loop, then continues with the code after that loop. Your program keeps running!

Tip

When to Use Break:

- Searching: Stop when you find the target
- User input: Exit when user chooses to quit
- Error conditions: Leave loop if something goes wrong
- Optimization: Don't check remaining items once done

Quick Check

What will this code print?

```
1 count = 0
2 while count < 10:
3     if count == 3:
4         break
5     print(count)
6     count = count + 1
7 print("Final count:", count)
```

Trace it: What's the last number printed before "Final count"?

3 VIDEO 2: The Continue Statement

3.1 Understanding Continue

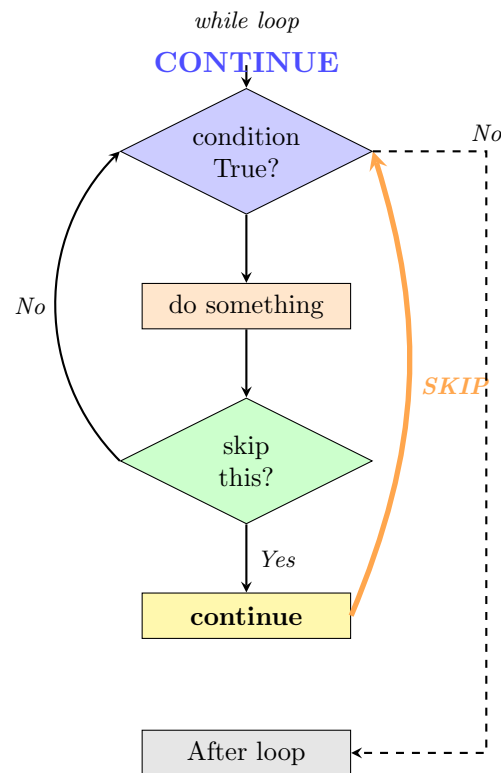
While **break** is the emergency exit, **continue** is more like a "skip turn" card. It doesn't exit the loop - it just skips the rest of the current iteration and jumps back to the top for the next one. Perfect for filtering or ignoring certain cases!

3.2 Your First Continue Statement

```
1 # Print only even numbers from 1 to 10
2 for num in range(1, 11):
3     if num % 2 != 0: # If odd
4         continue # Skip to next iteration
5     print(f"Even: {num}")
6 print("Done processing")
```

3.3 How Continue Works

Following is a flow diagram illustrating how the **continue** statement affects loop execution. As you can see, when Python hits **continue**, it jumps back to the start of the loop for the next iteration, skipping any remaining code in the current loop body.



Goes back to start

3.4 Continue vs Break Comparison

The **break** and **continue** statements are both used to alter the flow of loops, but they do so in different ways. Here's a side-by-side comparison:

Using Break:

```

1 # Stop at first multiple of 5
2 for num in range(1, 20):
3     if num % 5 == 0:
4         print(f"Found: {num}")
5         break
6     print(num)
7 # Output: 1, 2, 3, 4, Found: 5
    
```

Using Continue:

```

1 # Skip multiples of 5
2 for num in range(1, 20):
3     if num % 5 == 0:
4         continue
5     print(num)
6 # Output: 1,2,3,4,6,7,8,9,11...
    
```

3.5 Continue in While Loops

Be careful with **continue** in **while** loops - make sure you update your loop variable first!

```

1 # Process only even numbers up to 10
2 count = 0
3 while count < 10:
4     count = count + 1 # UPDATE FIRST!
5     if count % 2 != 0:
6         continue # Skip odd numbers
7     print(f"Even: {count}")
    
```

⚠ Common Error

Common Mistake: Using continue before updating the loop variable

```
1 # WRONG - Creates infinite loop!
2 count = 0
3 while count < 10:
4     if count % 2 != 0:
5         continue # Skips the increment!
6     count = count + 1 # Never reached for odd numbers
```

Always update your loop variable BEFORE using continue in while loops!

💡 Rule

Break vs Continue:

- break = "I'm done, exit the loop!"
- continue = "Skip this one, try the next!"

Think of break as STOP and continue as SKIP.

👉 Quick Check

What numbers will be printed?

```
1 for i in range(6):
2     if i == 2 or i == 4:
3         continue
4     print(i)
```

Which iterations get skipped?

4 VIDEO 3: Advanced Loop Control Patterns

4.1 Understanding Complex Control Flow

Now that you know `break` and `continue`, let's see how to combine them with other techniques to create sophisticated program control. These patterns appear everywhere in real software!

4.2 Pattern 1: Menu Systems

```
1 # Professional menu with multiple exit points
2 while True:
3     print("\n1. Play  2. Score  3. Help  4. Quit")
4     choice = input("Choose: ")
5
6     if choice == "4":
7         break # Clean exit
8     elif choice == "1":
9         print("Playing...")
10    elif choice not in "1234":
11        print("Invalid choice!")
12        continue # Skip to ask again
13    # Process valid choices here
```

4.3 Pattern 2: Number Guessing with Early Exit

```

1 # Guess the number with limited tries
2 secret = 42
3 max_tries = 5
4
5 for attempt in range(1, max_tries + 1):
6     guess = int(input(f"Guess (attempt {attempt}): "))
7
8     if guess == secret:
9         print("Correct! You win!")
10        break # Exit early on success
11    elif guess < secret:
12        print("Too low!")
13    else:
14        print("Too high!")
15 else: # Runs if loop completes without break
16     print(f"Sorry! The number was {secret}")

```

4.4 Pattern 3: Input Validation with Continue

```

1 # Sum positive numbers only
2 total = 0
3 count = 0
4
5 while count < 5:
6     num = int(input("Enter a positive number: "))
7
8     if num <= 0:
9         print("Must be positive! Try again.")
10        continue # Don't count this attempt
11
12    total = total + num
13    count = count + 1
14    print(f"Added! Sum so far: {total}")
15
16 print(f"Final sum: {total}")

```

4.5 Pattern 4: Nested Loop Control

Sometimes you need to control multiple loops. Here's a clean way using flags:

```

1 # Multiplication practice with rounds
2 quit_game = False
3
4 for round_num in range(1, 4): # 3 rounds
5     if quit_game:
6         break
7
8     print(f"\n=== Round {round_num} ===")
9
10    for problem in range(1, 4): # 3 problems per round
11        answer = input(f"Problem {problem} (or 'quit') ")
12
13        if answer == "quit":
14            quit_game = True
15            break # Exit inner loop

```

```

16     elif answer == "skip":
17         continue # Skip this problem
18
19     print(f"You answered: {answer}")

```

Tip

Clean Code Tips:

- Use `while True:` with `break` for menu loops
- Place `continue` early to reduce nesting
- Use flags to communicate between nested loops
- Always handle the exit condition explicitly

Quick Check

In the menu system pattern, why do we use `while True:` instead of a condition? What makes `break` cleaner here than a flag variable?

5 VIDEO 4: Debugging Infinite Loops

5.1 Understanding Infinite Loops

An infinite loop occurs when your loop condition never becomes false. With `while` loops, this is the no. 1 bug to watch for. Let's learn systematic debugging techniques to identify and fix them!

5.2 Debugging Technique 1: Print Statements

```

1 # Add debug prints to see what's happening
2 count = 10
3 target = 0
4 iterations = 0
5
6 while count != target:
7     print(f"DEBUG: count={count}, target={target}")
8     count = count - 3 # Problem: skips over 0!
9
10    iterations = iterations + 1
11    if iterations > 10: # Safety net
12        print("ERROR: Loop ran too long!")
13        break

```

5.3 Debugging Technique 2: Step-by-Step Tracing

```

1 # Trace with input() to pause each iteration
2 value = 1
3 while value < 100:
4     print(f"Current value: {value}")
5     input("Press Enter to continue...") # Pause!
6
7     value = value * 2

```



```

8
9     if value == 64: # Add checkpoint
10        print("Reached 64, still going...")

```

5.4 Common Infinite Loop Patterns

Pattern 1: The Skip-Over Problem

```

1 # WRONG: Never equals exactly 10
2 x = 0
3 while x != 10:
4     x = x + 3 # Goes: 0, 3, 6, 9, 12... skips 10!
5
6 #-----
7
8 # FIX: Use inequality instead
9 x = 0
10 while x < 10:
11     x = x + 3 # Stops when x >= 10

```

Pattern 2: The Continue Trap

```

1 # WRONG: Continue prevents update
2 count = 0
3 while count < 5:
4     if count == 2:
5         continue # Skips increment, stuck at 2!
6     count = count + 1
7
8 #-----
9
10 # FIX: Update before continue
11 count = 0
12 while count < 5:
13     count = count + 1 # Update first
14     if count == 3:
15         continue

```

⚠ Common Error

Emergency Exit Strategy:

If your program seems frozen:

1. Press Ctrl+C to stop the program
2. Add a debug print at the loop start
3. Add an iteration counter with maximum limit
4. Check: Is your variable updating? In the right direction? Can it reach the target?

5.5 Prevention Checklist

Before Running	Check This
Loop variable initialized?	<code>count = 0</code> before loop
Variable updates in loop?	<code>count = count + 1</code> inside
Update before continue?	Increment THEN continue
Condition reachable?	Will variable reach target value?
Exit condition clear?	Not too complex to understand
Safety limit added?	Max iterations or safety break

Quick Check

Debug this loop - why is it infinite?

```

1 num = 5
2 while num > 0:
3     if num == 2:
4         continue
5     print(num)
6     num = num - 1

```

Hint: What happens when num becomes 2?

6 VIDEO 5: Summary

6.1 What We Learned This Week

You've mastered the art of precise loop control! With **break** and **continue**, you can now exit loops exactly when needed, skip unwanted iterations, and create sophisticated control flows. You've learned to build professional menu systems, implement search algorithms with early exit, and most importantly - debug and prevent infinite loops!

6.2 Key Concepts We Covered

Concept	Purpose	Key Insight
break	Exit loop immediately	Only exits current loop
continue	Skip to next iteration	Loop keeps running
Infinite loops	When condition never false	Add safety limits
Debug prints	Track loop behavior	See variable values
Loop patterns	Menu/search/validation	Combine techniques
Safety nets	Prevent runaway loops	Max iteration counters

6.3 Your New Powers

- **Precision Control:** Exit or skip exactly when you want
- **Clean Menus:** Professional `while True` + `break` pattern
- **Smart Searches:** Stop looking once you find the answer
- **Robust Code:** Handle errors and prevent infinite loops
- **Debug Skills:** Systematically find and fix loop problems

★ Landmark Moment

You're now a loop control expert! You can make loops do exactly what you want - stop when needed, skip when appropriate, and never get stuck in infinite loops. Your programs can now handle complex user interactions, efficient searches, and robust error handling. You're thinking like a real programmer!

7 Quick Reference

📖 Quick Reference

Week 2 Essential Patterns:

Break Statement:

- Pattern: `if condition: break`
- Common use: Early exit when found/done
- Common mistake: Thinking it exits program (only exits loop!)

Continue Statement:

- Pattern: `if skip_condition: continue`
- Common use: Filter/skip unwanted iterations
- Debug tip: Update variables BEFORE continue

Menu Pattern:

- Pattern: `while True: ... if quit: break`
- Common use: User interfaces, game loops
- Common mistake: Forgetting the break condition

Infinite Loop Prevention:

- Pattern: Add iteration counter with limit
- Debug: Print loop variables each iteration
- Emergency: Ctrl+C to stop frozen program

7.1 Reflection Questions

1. Think about a game you play - where might it use `break` to exit a loop early? Where might it use `continue` to skip something?
2. Why is `while True: with break` sometimes cleaner than using a flag variable? When would a flag be better?
3. Describe a time this week when you created an infinite loop. What was the cause - missing update, wrong condition, or continue blocking update?
4. Look at the menu pattern. How would you modify it to count how many invalid choices the user makes?

5. If `break` only exits the current loop, how would you exit from deeply nested loops all at once?
(Hint: Think about flag variables!)

Next week: We'll dive into nested while loops and complex loop conditions, building on your new mastery of loop control!